

STUDENT CENTERED LEARNING

IMPLEMENTASI SPADE

Dosen Pengampu : Yutika Amelia Effendi, Ph.D.
Yayan Prima Nugraha, M.T.
Mata Kuliah : RKK307 Sistem Multi Agen
Prodi : Teknik Robotika dan Kecerdasan Buatan
Kelas : RK-A1

Uji Coba Kompleks SPADE (XMPP)

Tema: *Decentralized Task Allocation with Contract Net + Reputation + Fault Tolerance*

Output: Kode, log eksperimen, laporan simulasi

Tujuan Pembelajaran

Mahasiswa mampu:

1. Mengonfigurasi SPADE (Python, XAMPP) untuk banyak agen sekaligus (≥ 4).
2. Mengimplementasikan Contract Net Protocol (CNP) dengan FSMBehaviour (announce $\rightarrow$ bid $\rightarrow$ award $\rightarrow$ execute $\rightarrow$ report).
3. Memakai MessageTemplate dan performative (CALL_FOR_PROPOSAL/PROPOSE/ACCEPT/REJECT/INFORM/FAILURE).
4. Mengintegrasikan sistem reputasi (trust score) dan deadline (timeout dan penalty).
5. Menerapkan fault-tolerance (retry, re-announcement jika worker gagal/timeout).
6. Melakukan eksperimen: variasi jumlah tugas, worker heterogen (kapabilitas, biaya, latency), dan membandingkan metrik (waktu, keberhasilan, biaya total).

Arsitektur Sistem

- DirectoryAgent (DA): menyimpan registry JID seluruh agen; menyediakan service discovery simple.
- ManagerAgent (MA): mengumumkan tugas (CNP), memilih pemenang berdasarkan utility (gabungan penawaran dan reputasi), memonitor eksekusi, reassign jika gagal.
- WorkerAgent (WA) (≥ 3 agen): menerima CFP, menghitung proposal (waktu/biaya/keandalan), mengeksekusi tugas, mengirim INFORM/FAILURE.
- MonitorAgent (MonA): menerima event (log) dari MA/WA untuk audit dan perhitungan metrik global (opsional tapi direkomendasikan).

Spesifikasi Protokol (Wajib)

1. **FSM Behaviour Manager (MA)**
 - ANNOUNCE: MA kirim CFP ke semua WA terdaftar (payload JSON: {task_id, deadline, complexity, reward}).
 - COLLECT_BIDS (timeout T): MA kumpulkan PROPOSE dari WA, simpan ke daftar; jika timeout, lanjut ke SELECT.

- SELECT: $\text{skor} = w_{\text{proposal}} * f(\text{cost}, \text{duration}) + w_{\text{rep}} * \text{rep}(\text{worker})$ (Ibu boleh tetapkan bobot default: $w_{\text{proposal}}=0.7$, $w_{\text{rep}}=0.3$).
 - AWARD: kirim ACCEPT ke pemenang; REJECT ke lainnya.
 - EXECUTION_MONITOR (deadline D): tunggu INFORM/FAILURE; jika timeout/gagal → REASSIGN (kembali ke ANNOUNCE untuk worker lain).
 - REPORT: kirim ringkasan ke MonA; update reputasi pemenang (+) atau penalti (-).
2. **Behaviour Worker (WA)**
- Saat terima CFP:
 - Hitung proposal (misal: $\text{cost} = \text{base_cost}/\text{skill}$, $\text{duration} = \text{base_time}/\text{skill} + \text{network_delay}$).
 - Kirim PROPOSE (body JSON).
 - Saat terima ACCEPT:
 - Eksekusi *simulasi tugas* (sleep atau komputasi ringan sesuai duration).
 - Peluang gagal p (misal 10–20%) untuk menguji fault-tolerance (disimulasikan).
 - Kirim INFORM bila sukses, FAILURE bila gagal.
3. **Reputasi (Trust System)**
- Setiap WA punya skor awal $\text{rep}=0.5$.
 - Update: sukses $\text{rep} += +\Delta$ (mis. $+0.05$), gagal/timeout $\text{rep} += -\Delta$ (mis. -0.1), clamped $[0,1]$.
 - Skor dipakai MA saat SELECT.
4. **Fault-Tolerance dan Deadline**
- Timeout bid (pengumpulan proposal) dan timeout eksekusi (deadline).
 - Jika gagal/timeout → re-announce ke kandidat lain (yang belum pernah menang untuk task itu).
 - MA wajib menyimpan riwayat assignment per task agar tidak loop.
5. **Logging dan Monitor**
- Semua performative dan status dicatat (timestamped).
 - MonA menerima event via INFORM khusus log (topic log/...) untuk agregasi metrik.

Payload Standar (JSON)

- CFP: `{"task_id": "T12", "complexity": 3, "deadline": 12.0, "reward": 15.5}`
- PROPOSE: `{"task_id": "T12", "bid": {"cost": 7.2, "duration": 5.5}, "worker": "<jid>" }`
- ACCEPT/REJECT: `{"task_id": "T12"}`
- INFORM: `{"task_id": "T12", "status": "done", "actual_time": 5.4}`
- FAILURE: `{"task_id": "T12", "status": "failed", "reason": "timeout|exception"}`

Skenario Eksperimen

1. Skenario A (Baseline): 3 tugas, 3 worker homogen (skill sama).
2. Skenario B (Heterogen): 5 tugas, 4 worker (skill dan latency berbeda).

3. Skenario C (Stress + Fault): 10 tugas, 5 worker, $p_{fail} \approx 0.2$, deadline ketat (beberapa harus reassign).

Kumpulkan metrik per skenario:

- Persentase tugas selesai sebelum deadline.
- Rata-rata jumlah reassign per tugas.
- Waktu penyelesaian total (makespan).
- Biaya total (sum bid winner).
- Perubahan reputasi tiap worker.

Skeleton Kode (Contoh Petrified, dengan TODO)

Catatan: ini hanya kerangka dan mahasiswa wajib melengkapi bagian #TODO. Tidak harus menggunakan kerangka ini, dapat dieksplorasi sesuai kebutuhan.

common.py

```
import json
from spade.message import Message
PROTO = "cnp"
def make_msg(to, sender, performative, body_dict, thread=None):
    msg = Message(to=to)
    msg.set_metadata("performative", performative)
    msg.set_metadata("protocol", PROTO)
    if thread:
        msg.thread = thread
    msg.body = json.dumps(body_dict)
    return msg
```

directory_agent.py (DA)

```
from spade import agent, behaviour
class DirectoryAgent(agent.Agent):
    def __init__(self, jid, password):
        super().__init__(jid, password)
        self.registry = {"workers": set(), "managers": set(), "monitors": set()}

    class RegisterBehaviour(behaviour.CyclicBehaviour):
        async def run(self):
            msg = await self.receive(timeout=5)
            if msg and msg.metadata.get("performative") == "REQUEST" and msg.body:
                #TODO: parse role from body, update registry
                #reply with roster
                pass
    async def setup(self):
        self.add_behaviour(self.RegisterBehaviour())
```

worker_agent.py (WA)

```
import json, random, asyncio, time
from spade import agent, behaviour
from spade.template import Template
from common import make_msg, PROTO

class WorkerAgent(agent.Agent):
    def __init__(self, jid, password, skill=1.0, net_delay=0.2,
p_fail=0.1):
        super().__init__(jid, password)
        self.skill = skill
        self.net_delay = net_delay
        self.p_fail = p_fail
        self.rep = 0.5

    class CFPReceiver(behaviour.CyclicBehaviour):
        async def run(self):
            msg = await self.receive(timeout=5)
            if not msg: return
            if msg.metadata.get("protocol") == PROTO and
msg.metadata.get("performative") == "CALL_FOR_PROPOSAL":
                cfp = json.loads(msg.body)
                #TODO: compute bid using self.skill, self.net_delay
                bid = {"cost": 1.0/self.skill, "duration": 1.0/self.skill
+ self.net_delay}
                await self.send(make_msg(msg.sender, self.agent.jid,
"PROPOSE",
{"task_id": cfp["task_id"],
"bid": bid, "worker": str(self.agent.jid)},
thread=msg.thread))

            elif msg.metadata.get("performative") == "ACCEPT_PROPOSAL":
                data = json.loads(msg.body)
                # TODO: simulate execution with failure chance
                await asyncio.sleep(#duration based on accepted bid or
recompute)

                if random.random() < self.agent.p_fail:
                    #FAILURE
                    await self.send(make_msg(msg.sender, self.agent.jid,
"FAILURE",
{"task_id": data["task_id"],
"status":"failed","reason":"simulated"},
thread=msg.thread))

                else:
                    #INFORM
                    await self.send(make_msg(msg.sender, self.agent.jid,
"INFORM",
{"task_id": data["task_id"],
"status":"done","actual_time": 0.0},
```

```
thread=msg.thread))
```

```
async def setup(self):
    t = Template()
    t.metadata = {"protocol": PROTO}
    self.add_behaviour(self.CFPReceiver(), t)
```

manager_agent.py (MA)

```
import json, asyncio, time
from spade import agent, behaviour
from spade.template import Template
from common import make_msg, PROTO

def utility(bid_cost, bid_dur, rep, w_prop=0.7, w_rep=0.3):
    #TODO: design utility (lower better or higher better; konsisten!)
    score_prop = 1.0 / (1e-6 + bid_cost + bid_dur)
    score = w_prop * score_prop + w_rep * rep
    return score

class ManagerAgent(agent.Agent):
    def __init__(self, jid, password, workers, monitor=None,
bid_timeout=5, exec_timeout=10):
        super().__init__(jid, password)
        self.workers = workers
        self.monitor = monitor
        self.bid_timeout = bid_timeout
        self.exec_timeout = exec_timeout
        self.reputation = {w: 0.5 for w in workers} #worker JID -> rep
        self.assigned = {} #task_id -> winner

class CNPBehaviour(behaviour.FSMBehaviour):
    async def on_start(self):
        self.set("task", self.get("task") or {})
        self.set("bids", [])

class AnnounceState(behaviour.State):
    async def run(self):
        task = self.get("task")
        thread = f"TASK-{task['task_id']}"
        for w in self.agent.workers:
            await self.send(make_msg(w, self.agent.jid,
"CALL_FOR_PROPOSAL", task, thread=thread))
        self.set_next_state("COLLECT")

class CollectBidsState(behaviour.State):
    async def run(self):
        start = time.time()
        while time.time() - start < self.agent.bid_timeout:
            msg = await self.receive(timeout=0.5)
```

```

        if msg and msg.metadata.get("performative") == "PROPOSE":
            self.get("bids").append((str(msg.sender),
json.loads(msg.body)["bid"]))
            self.set_next_state("SELECT")

class SelectState(behaviour.State):
    async def run(self):
        task = self.get("task")
        bids = self.get("bids")
        #TODO: choose winner by utility using self.agent.reputation
        if not bids:
            #re-announce or abort
            self.set_next_state("ANNOUNCE"); return
        best = None; best_score = -1
        for sender, bid in bids:
            rep = self.agent.reputation.get(sender, 0.5)
            score = utility(bid["cost"], bid["duration"], rep)
            if score > best_score:
                best, best_score = (sender, bid), score
        winner, wbid = best
        self.agent.assigned[task["task_id"]] = winner
        await self.send(make_msg(winner, self.agent.jid,
"ACCEPT_PROPOSAL", {"task_id": task["task_id"]}, thread=f"TASK-
{task['task_id']}"))
        #Reject others
        for sender, bid in bids:
            if sender != winner:
                await self.send(make_msg(sender, self.agent.jid,
"REJECT_PROPOSAL", {"task_id": task["task_id"]}, thread=f"TASK-
{task['task_id']}"))
        self.set("winner", winner)
        self.set_next_state("EXEC_MON")

class ExecMonitorState(behaviour.State):
    async def run(self):
        task = self.get("task")
        start = time.time()
        while time.time() - start < self.agent.exec_timeout:
            msg = await self.receive(timeout=0.5)
            if not msg: continue
            if msg.metadata.get("performative") == "INFORM":
                #update rep +
                self.agent.reputation[str(msg.sender)] = min(1.0,
self.agent.reputation.get(str(msg.sender),0.5) + 0.05)
                self.set_next_state("REPORT"); return
            if msg.metadata.get("performative") == "FAILURE":
                #update rep -
                self.agent.reputation[str(msg.sender)] = max(0.0,
self.agent.reputation.get(str(msg.sender),0.5) - 0.1)
                #reassign:
                self.set("bids", [(s,b) for (s,b) in self.get("bids")
if s != str(msg.sender)])

```

```

        self.set_next_state("SELECT"); return
    #timeout
    self.set_next_state("SELECT")

class ReportState(behaviour.State):
    async def run(self):
        #TODO: send log to MonitorAgent if exists
        self.kill() # selesai untuk task itu

    async def setup(self):
        pass #mahasiswa inisialisasi tasks dan menambahkan state
        transitions

```

monitor_agent.py (MonA) – opsional

```

from spade import agent, behaviour
from spade.template import Template

class MonitorAgent(agent.Agent):
    def __init__(self, jid, password):
        super().__init__(jid, password)
        self.events = []

    class LogReceiver(behaviour.CyclicBehaviour):
        async def run(self):
            msg = await self.receive(timeout=5)
            if msg and msg.metadata.get("protocol") == "log":
                self.agent.events.append((msg.sender, msg.body))

    async def setup(self):
        t = Template()
        t.metadata = {"protocol": "log"}
        self.add_behaviour(self.LogReceiver(), t)

```

Jika mahasiswa menggunakan skeleton di atas, mahasiswa wajib:

- Menyambungkan DA ↔ MA/WA (registrasi dan pemberian roster).
- Melengkapi seluruh #TODO.
- Menambah *unit test* sederhana atau skrip runner yang meluncurkan semua agen dan 1 set tugas.

Panduan Akun dan Konfigurasi

- Buat akun XMPP berbeda untuk setiap agen (contoh: ma@jabber.at, wa1@jabber.at, wa2@jabber.at, wa3@jabber.at, da@jabber.at, mon@jabber.at).
- Simpan kredensial dalam .env atau config.json.
- Jalankan tiap agen pada terminal terpisah.

Eksperimen yang Harus Dilaporkan

1. Baseline vs Heterogen vs Stress+Fault (*lihat Skenario*).
2. Grafik/ tabel:
 - Task success rate vs deadline.
 - Rata-rata reassign per task.
 - Biaya total sistem.
 - Evolusi reputasi per worker.
3. Analisis:
 - Dampak bobot reputasi (w_{rep}) pada pemilihan pemenang.
 - Dampak p_{fail} pada waktu total dan reassign.
 - *Trade-off* biaya vs reputasi.

Yang Dikumpulkan

1. Dikerjakan secara berkelompok
2. Code program (folder proyek) + `requirements.txt`.
3. Script run (`run_all.sh / run_all.py`) untuk meluncurkan semua agen.
4. Log (raw + ringkasan metrik).
5. Laporan PDF format bebas yang berisi arsitektur, protokol, formula utility, konfigurasi eksperimen, hasil dan analisis.
6. Video demo singkat.

Selamat Mengerjakan